

But if you're planning for Postgres in production, you should use the same in development too.

How containers can help

Okay, you want to experiment with multiple MySQL or Postgres projects but still don't want to have a difficult time managing them. Containerisation comes to your rescue.

The idea is to create a container whenever you need a new database for a new project. You can stop the container or even delete it when you are done with it. No installations, no administration. You can't get closer to the ease of using SQLite with DB servers.

Do you need more explanation for the no-installations-no-administration part? For Docker, there are pre-built images for services like Postgres and MySQL. Once you've downloaded such an image, you can spawn as many instances as you want. Since each project gets its own DB server container, the amount of work required to set up the database, roles and permissions is trivial.

 **Note:** The only intention of this article is to serve as an introduction to Docker and Docker Compose while sharing a solution to some problems faced by database application developers. Docker is a great development tool. But there is a certain amount of risk when using it in production, especially in terms of security and performance. You should use Docker in production only if there is a real need and you have to be extra careful when you do so.

Installing Docker

Installing Docker used to be a bit difficult, but not anymore. All you have to do in Ubuntu 20.04 is run these commands:

```
sudo apt update
sudo apt install docker.io docker-compose
```

Don't forget to add yourself to the Docker group:

```
sudo usermod -aG docker YOUR_USERNAME
```

You'll have to log out and log in again to apply the group change, but that's all it takes. Now you can use this Docker setup to run as many containers and projects as you want. However, remember to monitor the containers and clean up the junk frequently, or your system resources will be wasted.

Let's start the container!

Now that you have Docker installed on your machine, you can run this command to start a Postgres instance:

```
docker run --name mypostgres1 -e POSTGRES_PASSWORD=mypgpass
```

```
-d postgres
```

It'll pull the Docker image from the Internet automatically and start an instance of it. Pulling is a one-time process, which means subsequent runs will be faster.

In the above command, *mypostgres1* is a name of your choice, which you can use in commands like:

```
docker stop mypostgres1
```

The flag *-e* sets an environment variable, and here you can use it to set the *POSTGRES_PASSWORD* to *mypgpass*, the password of your choice. *-d* means detached, which will cause the container to run in the background, allowing you to use the shell for other purposes. Finally, *postgres* at the end of the command is the image name.

To enter the Postgres shell of this instance, run the following command:

```
docker exec -it mypostgres1 psql -U postgres
```

But this setup is only useful to try out some SQL commands. To run a real project, you have to manage ports, persist data, etc. It's time to get introduced to Docker Compose.

Docker Compose

Docker Compose is an abstraction over Docker that makes life easier when you need more control over your containers like setting environment variables, persisting data, coordinating multiple services (e.g., database server and Web server), etc. With Compose, you can define all your services in a human-readable YAML file and run the command *docker-compose* with minimal arguments to get them started. It's still possible to manage containers started this way using the native Docker commands.

Postgres with Compose

Let's first create a working directory with the following structure:

```
- myproject/
  - initdb.d/
  - pgdatadir/
  - docker-compose.yml
  - pg.env
```

initdb.d is a folder to keep the SQL files that are to be executed automatically while initialising the database. *pgdatadir* is an empty folder in which you ask your Postgres instance to store all the data. Both are optional.